Software Programming

IGS1931

Week 9: Strings

Review, Debug & Practice



HK©2026

Today

Review	Debug	Write it
Predict, then run	Four broken functions	5 easy, 2 medium, 1 hard
String operations	One bug each	Plus a bonus exercise

By the end of today

Read positive and negative indexes
Use [start:end:step], including empty slices
Build a new string when you need a change
Choose a method that fits the task

Quotes and escape characters

What will be printed?

```
print('"Hello, John", he said.')
print('Hello\nWorld')
print('a\tb\\c')
print(len('one\ntwo'))
```

```
"Hello, John", he said.
Hello
World
a       b\c
7
```

Write	Meaning
\' \"	A quote
\n / \t	New line / tab
\\	One backslash
'''...'''	Several lines

"don’t" uses double quotes around an apostrophe.

Indexing: one character with []

```
m = 'Welcome to INHA university'
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
W	**e**	**l**	**c**	**o**	**m**	**e**	**·**	**t**	**o**	**·**	**I**	**N**	**H**	**A**	**·**	**u**	**n**	**i**	**v**	**e**	**r**	**s**	**i**	**t**	**y**
-26																							-3	-2	-1

What will be printed?

`m[6]`	`i = 10` `m[i - 1]`	`m[-1]`	`m[-3]`	`m[26]`
`e`	`o`	`y`	`i`	`IndexError`

First index: 0 Last index: len(m) - 1 = 25

Slicing: [start:end]

```
m = 'Welcome to INHA university'
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
W	**e**	**l**	**c**	**o**	**m**	**e**	**·**	**t**	**o**	**·**	**I**	**N**	**H**	**A**	**·**	**u**	**n**	**i**	**v**	**e**	**r**	**s**	**i**	**t**	**y**
-26																							-3	-2	-1

What will be printed?

```
m[8:10]
```

```
to
```

```
m[14:]
```

```
A university
```

```
m[11:20]
```

```
INHA univ
```

```
m[:10]
```

```
Welcome to
```

```
m[-10:]
```

```
university
```

```
m[:-1]
```

```
Welcome to INHA universit
```

Start included. End excluded. Missing bounds use the string's edges.

Slicing with a step: [start:end:step]

```
m = 'Welcome to INHA university'
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
W	**e**	**l**	**c**	**o**	**m**	**e**	**·**	**t**	**o**	**·**	**I**	**N**	**H**	**A**	**·**	**u**	**n**	**i**	**v**	**e**	**r**	**s**	**i**	**t**	**y**
-26																							-3	-2	-1

```
m[0:7:3]
```

```
Wce
```

```
m[0:26:2]
```

```
Wloet NAuiest
```

```
m[9:7:-1]
```

```
ot
```

```
m[::-2]
```

```
yirvn HIo mce
```

```
m[0:7:-1]
```

```
''  (empty)
```

For [9:7:-1], visit 9 and 8. Stop before 7. s[::-1] reverses s.

Building strings: +, * and immutability

What will be printed?

```
a = m[:7]
b = m[10:15]
print(a + b)
word = m[11:15] + '\n'
print(word * 3)
```

```
Welcome INHA
INHA
INHA
INHA
```

```
fruit = 'bananas'
fruit[2] = 'N'  # TypeError
```

```
new_fruit = fruit[:2] + 'N' + fruit[3:]
# 'baNanas'
```

len, in and loops

What will be printed?

```
word = 'banana'
count = 0
for letter in word:
    if letter == 'a':
        count += 1
print(count)
print('ana' in word, 'Ban' in word,
      'x' not in word)
index = 0
while index < len(word):
    print(index, word[index])
    index += 2
```

```
3
True False True
0 b
2 n
4 n
```

for ch in s visits each character.

in is case-sensitive.

Methods and their return values

Method	Example and result
upper() / lower()	'Hi'.upper() → 'HI'
index(x)	'abracadabra'.index('c') → 4
count(x)	'abracadabra'.count('a') → 5
strip()	' a b '.strip() → 'a b'
replace(a, b)	'Hello'.replace('l', 'L') → 'HeLLo'
split(sep)	'Hello, World!'.split(',') → ['Hello', ' World!']

```
s = 'Hello, World!'
s.upper()
print(s)
```

```
Hello, World!
Keep the result: s = s.upper()
```

Comparing strings

```
'apple' < 'banana'
```

```
True
```

```
'Pineapple' < 'banana'
```

```
True
```

```
'Pineapple'.lower() <
'banana'.lower()
```

```
False
```

```
'cat' == 'Cat'
```

```
False
```

Python compares Unicode code points from left to right.
For English A–Z and a–z, uppercase letters sort first.

Bug hunt: four classic string bugs

1

```
return s[len(s)]
```

IndexError

```
return s[-1]
```

2

```
s[0] = s[0].upper()
```

TypeError

```
return s[0].upper() + s[1:]
```

3

```
s.upper()
```

Result discarded

```
return s.upper() + '!'
```

4

```
else: return -1
```

Stops after first mismatch

```
# after the loop:
return -1
```

String cheat sheet

Want	Code
First / last character	s[0] / s[-1]
First 3 / last 3	s[:3] / s[-3:]
Every other character / reverse	s[::2] / s[::-1]
Middle position	half = len(s) // 2
Replace a character	s[:i] + new + s[i+1:]
Membership	x in s
Each word	for w in s.split():
Build a result	result = '' then result += ...
Digit test	ch in '0123456789'

s[len(s)] raises IndexError. Save returned strings.
Check x in s before using s.index(x).

Quick check: what will be printed?

```
s = 'INHA2026'
print(s[4:] + s[:4])
```

```
2026INHA
```

```
print('banana'.replace('a', '')[::-1])
```

```
nnb
```

```
w = 'Python'
print(w[-1:-4:-1])
```

```
noh
```

```
print(len('a\nb'), 'IGS' < 'igs')
```

```
3 True
```

Exercise time

Open Colab from the README’s Week 9 badge

Run cells in order. Write a prediction before you run.

Level	Exercise	Example result
Easy 1	admission_year('12240513')	'2024'
Easy 2	mask_phone	'010-****-5678'
Easy 3	hidden_message('Hxexlxlxo')	'Hello'
Easy 4	is_mirror_time('12:21')	True
Easy 5	strong_password('inha2026')	True
Medium 1	clean_name(' aLiCe kIM ')	'Alice Kim'
Medium 2	swap_halves('abcde')	'decab'
Hard	expand('x12y2')	12 x characters, then 'yy'
Bonus	Exercise 7.1: firstparens	See notebook prompt